

University OF Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	Course Code: CMPE-112L
Assignment Type: lab	Dated: 18 MAY 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 10	CLOs to be covered: CLO2
Lab Title: RECURSION	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10

Lab Goals/Objectives:

- **Divide and Conquer:** It naturally maps to problems where a task can be divided into smaller, independent chunks of the exact same nature (e.g., searching a binary tree or sorting data).
- **Code Cleanliness & Elegance:** It eliminates the need for complex nested loops and temporary tracking variables, making the code much closer to mathematical logic.
- **Handling Dynamic/Unknown Depths:** It is the go-to strategy when you don't know how deep a structure goes ahead of time—like navigating folders inside folders on a computer, or traversing complex data structures like graphs and trees.

EQUIPMENT Required:

- Computer/Laptop
- Python 3.x
- VS Code / PyCharm / IDLE

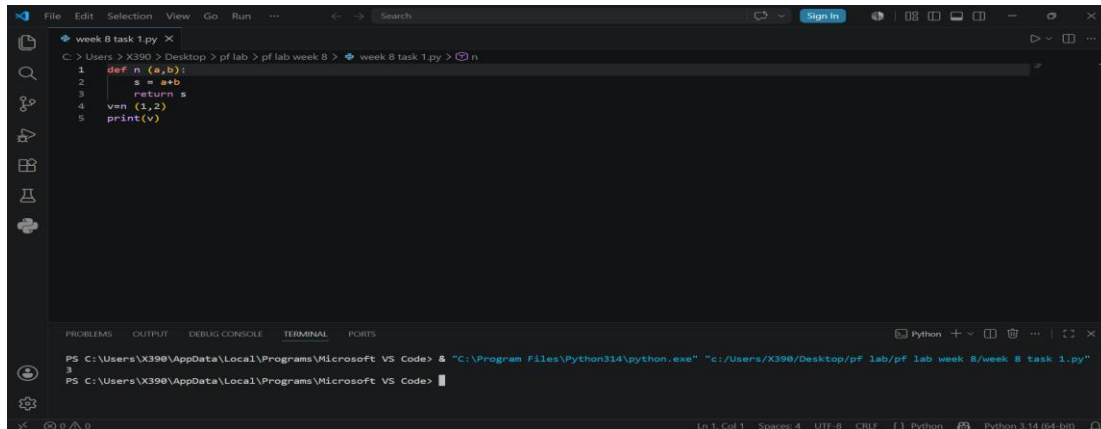
THEORY:

- **Recursion** is a programming technique where a function solves a problem by calling a smaller version of itself. It operates on the **divide-and-conquer** principle, breaking a complex task into identical, manageable sub-problems. Every valid recursive function requires a **base case** to act as an exit strategy and prevent infinite loops. It inherently utilizes the system's **call stack** to keep track of each active, unfinished function call in memory. While it provides highly elegant, clean code for traversing **hierarchical data structures** like trees and graphs, it can lead to stack overflow if it goes too deep.

TASK NO 1:

write a Python program for your Programming Fundamentals Lab (**week 8 task 1.py**) that demonstrates how to define a user-defined function, pass arguments to it, perform a basic arithmetic operation, and return a value.

RESULT:



The screenshot shows the Visual Studio Code editor with a file named 'week 8 task 1.py'. The code defines a function 'n(a,b)' that calculates the sum of 'a' and 'b', returns the result, and then calls the function with arguments (1,2) and prints the result. The terminal at the bottom shows the command to run the script and the output '3'.

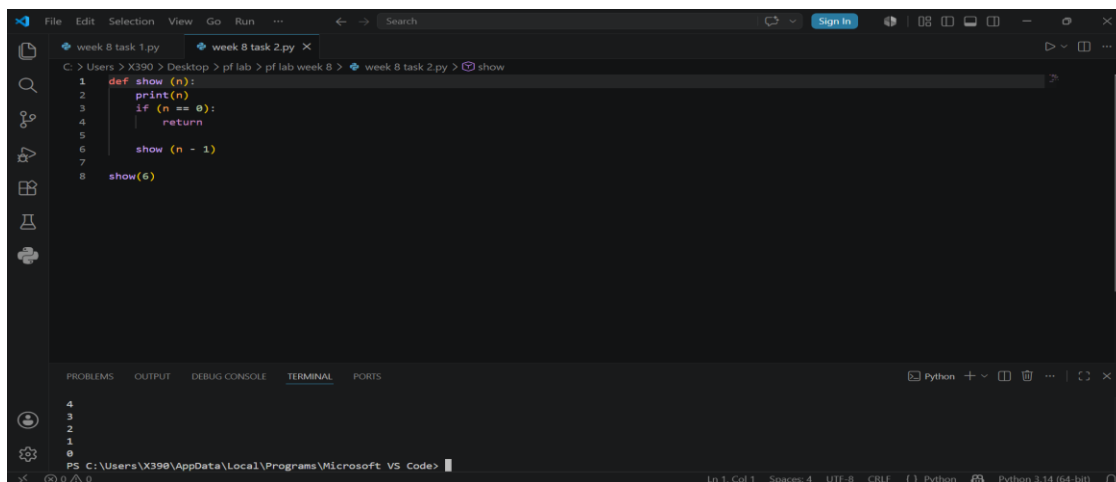
```
1 def n(a,b):
2     s = a+b
3     return s
4 v=n(1,2)
5 print(v)
```

```
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 8/week 8 task 1.py"
3
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 2:

Write a Python program for your Programming Fundamentals Lab (**week 8 task 2.py**) that demonstrates a foundational implementation of **recursion**. The objective is to understand how a function can call itself to print a sequence of numbers downward until a base case condition is met.

RESULT:



The screenshot shows the Visual Studio Code editor with a file named 'week 8 task 2.py'. The code defines a recursive function 'show(n)' that prints the value of 'n' and then calls itself with 'n-1' until it reaches the base case 'n == 0'. The terminal at the bottom shows the command to run the script and the output '4 3 2 1 0'.

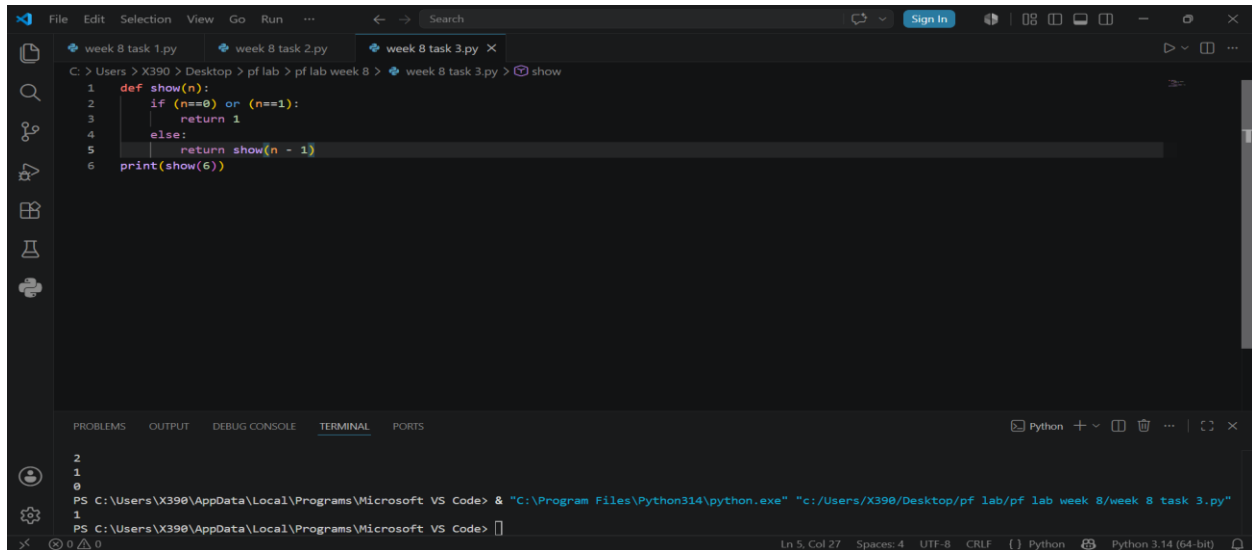
```
1 def show(n):
2     print(n)
3     if (n == 0):
4         return
5     show(n - 1)
6
7 show(4)
```

```
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 3:

Write a Python program for your Programming Fundamentals Lab (**week 8 task 3.py**) to demonstrate how recursive functions return values back up through the call stack. The goal is to implement a structural skeleton commonly used in mathematical sequence reductions, establishing clear conditional branching for base and recursive states.

RESULT:



The screenshot shows a Visual Studio Code editor window with a Python file named 'week 8 task 3.py'. The code defines a recursive function 'show(n)' that returns 1 for n=0 or n=1, and otherwise returns 'show(n-1)'. The function is called with 'show(6)'. The output in the terminal is '2 1 0 1', which represents the sequence of return values from the recursive calls: show(6) returns 2, show(5) returns 1, show(4) returns 0, and show(3) returns 1.

```
1 def show(n):
2     if (n==0) or (n==1):
3         return 1
4     else:
5         return show(n - 1)
6 print(show(6))
```

```
2
1
0
1
```

PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab week 8/week 8 task 3.py"

CONCLUSION:

- **Recursion** is ultimately a powerful mental model that shifts the approach from brute-force looping to elegant, self-referential problem-solving. Its ultimate success relies entirely on a well-defined **base case** to safely halt execution and unwind the memory stack. The technique shines brightest when handling complex, non-linear data structures like **trees and graphs** where nesting depths are unpredictable. However, developers must carefully weigh its structural beauty against the concrete risks of **high memory overhead** and potential stack overflows. In short, it transforms massive, daunting tasks into a chain of simple operations, serving as an indispensable tool for optimized algorithmic design